SKILLNAVORA · AI PRODUCT MANAGEMENT & GENAI LAUNCHPAD

Platform & API Product Management

Manage platforms and APIs as products: developer experience, API design decisions, versioning and limits, ecosystems and partners, platform metrics, and AI/MCP platforms.

Edition 2026 · skillnavora.in

How to use this guide

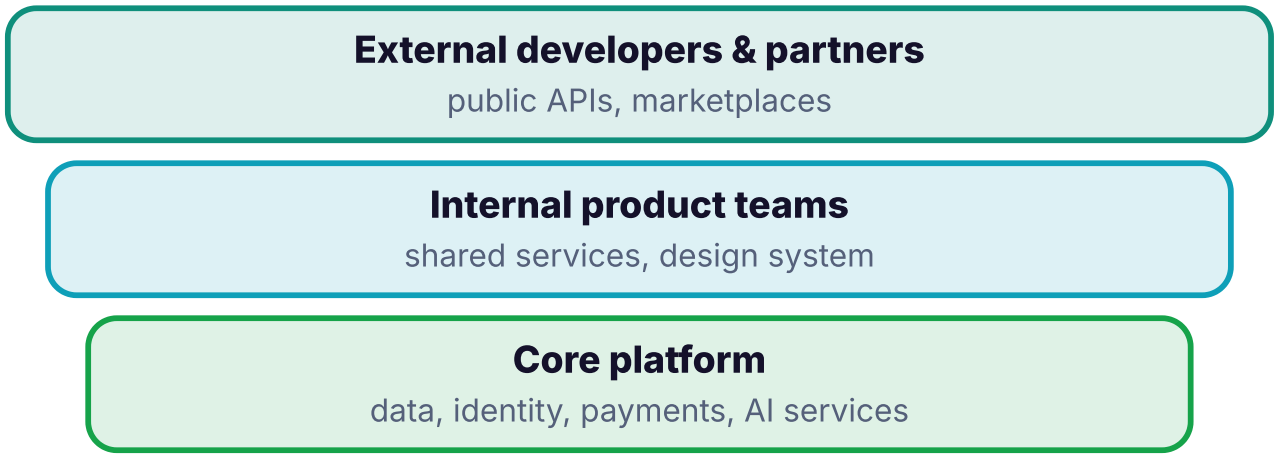
Each class has goals, concepts with Indian examples, key takeaways and a practice exercise, from basic to advanced. Finish with the final project to build something you can show in interviews.

	Class	Level
Class 1	**Platforms and APIs as products**	Basic
Class 2	**Developer experience (DX)**	Basic
Class 3	**API design decisions for PMs**	Intermediate
Class 4	**Ecosystems and partners**	Intermediate
Class 5	**API lifecycle, versioning and deprecation**	Intermediate
Class 6	**Developer portal, documentation and community**	Intermediate
Class 7	**API security, authentication and compliance**	Intermediate
Class 8	**Platform metrics**	Advanced
Class 9	**Platform business models and network effects**	Advanced
Class 10	**Internal platforms and platform engineering**	Advanced
Class 11	**Case studies: payments, communications and open networks**	Advanced
Class 12	**Case studies: three platform decisions**	Advanced
Class 13	**Interview practice: platform and API scenario questions**	Advanced
Class 14	**AI platforms and final project**	Advanced
	Toolkit: templates, cheat sheet, glossary, 14 interview questions, further reading	

BASIC · CLASS 1

Platforms and APIs as products

Understand platform types and users.



Platform layers

Examples: UPI/payment APIs (Razorpay, Stripe), maps APIs, WhatsApp Business API, internal identity or AI platforms, MCP servers that expose your product to AI assistants.

Product types compared

	Application	Platform / API
Customer	End user	Developers, partners and the apps they build
Value	Solves a task	Enables others to solve many tasks
Growth	Users and revenue	Ecosystem size, integrations, usage volume
Risk	Usability, retention	Breaking changes, reliability, trust
Roadmap	Features	Capabilities, stability, developer experience

Examples: Stripe and Razorpay (payments), Twilio (messaging), Google Maps Platform (location), UPI and ONDC (open networks), Shopify (commerce platform), and AI APIs from model providers.

Key takeaways

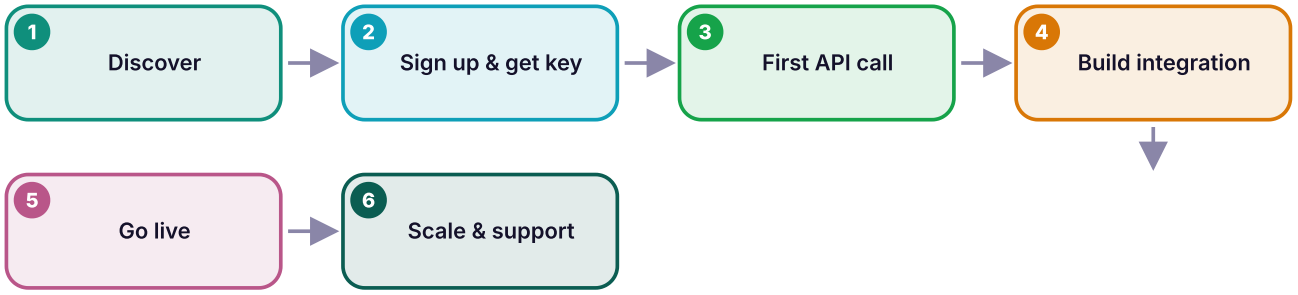
- Developers are the users.
- Internal platforms are products too.
- AI assistants are a new channel (MCP).

Practice exercise: Map the platform users and their jobs for a payments company.

BASIC · CLASS 2

Developer experience (DX)

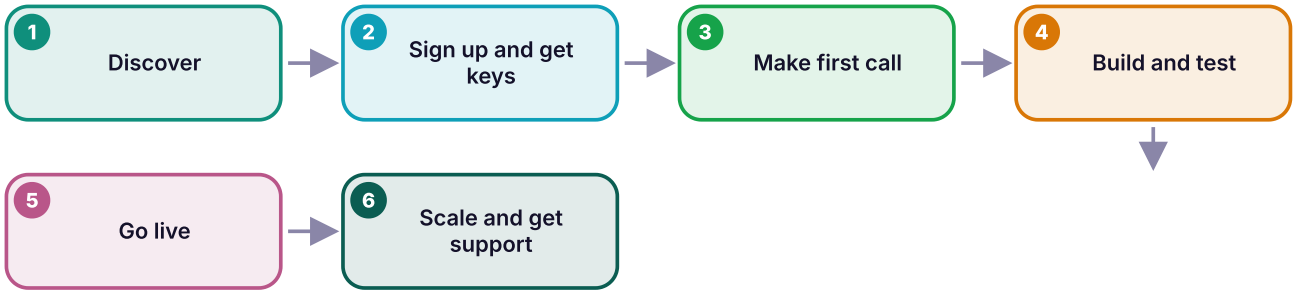
Make developers successful fast.



Developer journey

- Time to first successful call (aim for minutes).
- Clear docs, quickstarts, SDKs, sandbox keys, Postman collections.
- Helpful errors, status page, changelog, community support.

Developer journey



Developer experience funnel

The key metric is **time to first successful call** (TTFC). Under 10 minutes is a good target for many APIs. Every extra step (manual approval, unclear docs, missing sandbox) loses developers.

Key takeaways

- Measure time to first call.
- Docs and sandbox are the onboarding.
- Errors should explain the fix.

Practice exercise: Audit a public API's onboarding and list 5 improvements.

INTERMEDIATE · CLASS 3

API design decisions for PMs

Decide what matters in API contracts.

- Resources and endpoints that match developer jobs; consistent naming.
- Authentication (API keys, OAuth), permissions and scopes.
- Versioning and deprecation policy (e.g. 12 months notice).
- Rate limits and quotas; idempotency keys for payments.
- Webhooks for events; pagination; error formats.

API design checklist for PMs

Topic	Decision and why it matters
Resources and naming	Use customer-meaningful nouns (orders, payments); consistent patterns
REST vs GraphQL vs events	REST is simple and cacheable; GraphQL lets clients ask for exact data; webhooks and events notify of changes
Versioning	Never break existing clients; introduce versions (v1, v2) or date-based versions; publish a deprecation policy
Idempotency	Allow safe retries for payments and orders using idempotency keys
Pagination and filtering	Cursor pagination for large lists
Errors	Consistent codes, human-readable messages, request IDs
Rate limits and quotas	Protect the system; document limits and retry guidance
Auth and scopes	API keys for simple cases; OAuth 2.0 for user-delegated access; least-privilege scopes

Key takeaways

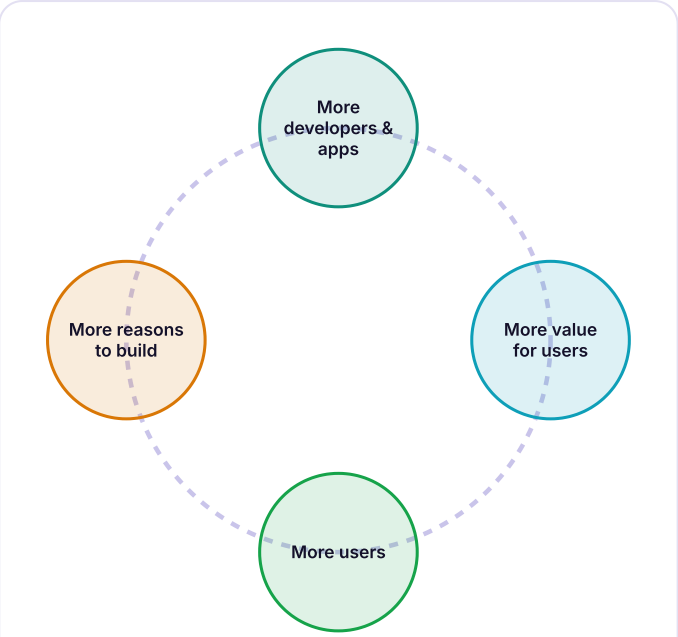
- Contracts are promises: version carefully.
- Rate limits protect reliability.
- Webhooks make integrations real-time.

Practice exercise: Write the API requirements for a 'refund' endpoint.

INTERMEDIATE · CLASS 4

Ecosystems and partners

Grow a platform through others.



Platform network effect

- Partner tiers, certification, marketplace listing, revenue share.
- Governance: app review, security, data access rules.

Partner programme tiers

Tier	Requirements	Benefits
Registered	Sign up, accept terms	Docs, sandbox, community
Certified	Pass quality and security checks	Listing, badge, support
Strategic	Revenue or usage thresholds, joint roadmap	Co-marketing, early access, account manager

Key takeaways

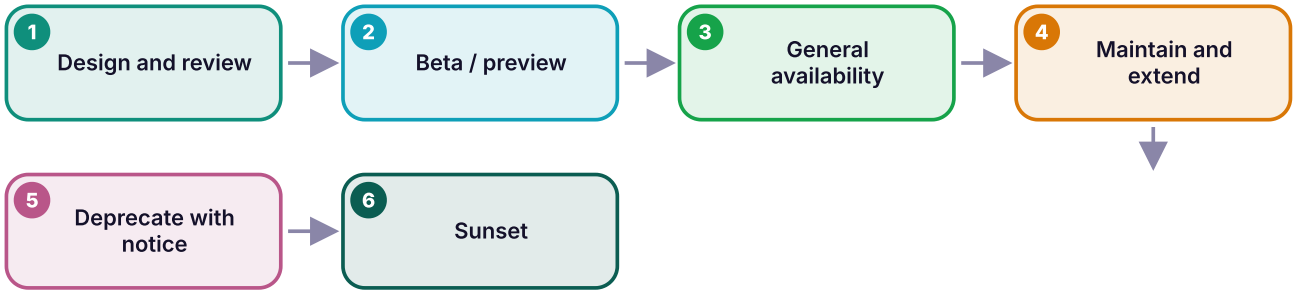
- Platforms grow through network effects.
- Partners need clear rules and incentives.
- Review apps for security.

Practice exercise: Design a partner program for a SaaS marketplace.

INTERMEDIATE · CLASS 5

API lifecycle, versioning and deprecation

Evolve APIs without breaking the apps that depend on them.



The API lifecycle

- **Contract first:** design using OpenAPI; review with real developers before building.
- **Breaking vs non-breaking:** adding optional fields is usually safe; removing or renaming fields, changing types or behaviour is breaking.
- **Deprecation policy:** announce early (6 to 12 months for critical APIs), publish migration guides, add warning headers, email active users, track usage of old versions, sunset only when usage is low or after contacting the remaining users.
- **Changelog and status:** a public changelog and a status page build trust.
- **SDKs and tooling:** official SDKs in top languages, Postman collections, CLI, and code samples that stay tested.
- **Backwards compatibility tests:** automated contract tests so a release cannot break clients silently.

> For a platform, a breaking change you did not announce can cost more trust than a bug. Treat deprecation as a product launch with its own plan.

Key takeaways

- Design contract-first and with developers.
- Deprecate with long notice and migration help.
- Automate compatibility tests.

Practice exercise: Write a deprecation plan for an API field you want to remove.

INTERMEDIATE · CLASS 6

Developer portal, documentation and community

Make developers successful without talking to you.

Element	What good looks like
Quickstart	Working code in 5 minutes, copy-paste, in the top 3 languages
Reference docs	Every endpoint with examples, errors, limits and try-it console
Guides and use cases	Task-based tutorials (accept a payment, send a message)
Sandbox and test data	Safe environment with fake data and test keys
Search and navigation	Fast search, versioned docs, copy buttons
Support and community	Forum, Discord or Slack, office hours, response-time targets
Status and changelog	Incidents, maintenance, release notes
Analytics	Docs page feedback, search terms with no results, drop-off in quickstart

Docs are product. Assign owners, test them with new developers, and measure time to first call, quickstart completion, docs satisfaction and support tickets per 1,000 developers.

Key takeaways

- Quickstart in minutes, in popular languages.
- Sandbox and test data are essential.
- Measure the docs funnel.

Practice exercise: Test your favourite API's quickstart as a new developer and list the three biggest frictions.

INTERMEDIATE · CLASS 7

API security, authentication and compliance

Protect users, data and the platform.

Topic	Plain meaning	PM decision
API keys	A secret string identifying the app	Rotation, scopes, per-environment keys
OAuth 2.0	Users grant apps limited access without sharing passwords	Which scopes? Consent screens and review of apps
JWT and tokens	Signed proof of identity and permissions with expiry	Token lifetime vs usability
Rate limiting and quotas	Caps per key or user	Fair use, abuse protection, paid tiers
Webhook signatures	Prove the message came from you	Documentation and retry rules
Data protection	Encryption, minimisation, retention	DPDP, GDPR, PCI-DSS, RBI rules
Audit logs	Who did what and when	Customer-visible logs for enterprise
App review	Check partner apps before they get sensitive scopes	Review criteria and timelines

Principle: **least privilege**. Give each app only the access it needs, make it visible to users, and make it easy to revoke.

Key takeaways

- Use OAuth and scopes for user data.
- Rate limit and sign webhooks.
- Review partner apps before granting sensitive access.

Practice exercise: Define scopes and a review process for an app marketplace that can read customer orders.

ADVANCED · CLASS 8

Platform metrics

Measure platform health.

Metric	Why
Active developers / apps	Adoption
Time to first call / to production	DX quality
API calls, success rate, latency	Usage and reliability
Revenue via platform	Business value
Support tickets per integration	Friction

Platform metrics tree

Layer	Metrics
Adoption	New developers, activation (first call), apps live, time to first call
Usage	API calls, active apps, requests per app, feature adoption
Quality	Uptime, p95/p99 latency, error rate, incident count and time to resolve
Satisfaction	Developer NPS, docs ratings, support CSAT, time to resolution
Business	Revenue, margin, ecosystem revenue, partner-sourced revenue, churn of apps
Health	Breaking-change incidents, deprecation completion, security findings

Key takeaways

- Mix adoption, DX, reliability and revenue.
- Track success rate and latency.
- Use tickets to find friction.

Practice exercise: Build a platform dashboard spec.

ADVANCED · CLASS 9

Platform business models and network effects

Choose how a platform earns and grows.

Model	How it earns	Example	Watch out for
Usage-based	Per call, message or transaction	Payments, SMS, maps, AI APIs	Unpredictable bills; margin on usage
Tiered subscription	Plan with included usage	Developer tools	Plan boundaries confuse
Revenue share / take rate	Percentage of partner revenue	Marketplaces and app stores	Partners resent high rates
Freemium plus enterprise	Free to start, paid for scale and support	Many API products	Cost of free usage
Data / network value	Value rises as more join	Open networks like UPI and ONDC	Needs governance and neutrality

Network effects

- **Direct:** more users make the product more valuable for each user.
- **Two-sided:** more developers attract more users and vice versa; solve the "chicken and egg" by subsidising one side or seeding with your own apps.
- **Data effects:** usage improves the platform (fraud models, routing).
- **Multi-homing:** developers may use competing platforms; lower switching for them means you must keep quality and economics strong.

Key takeaways

- Match pricing to cost and value.
- Solve the chicken-and-egg problem on purpose.
- Watch multi-homing and take-rate tension.

Practice exercise: Choose a business model for a delivery-logistics API and justify it.

ADVANCED · CLASS 10

Internal platforms and platform engineering

Treat internal tools as products for internal customers.

- **Platform as a product:** internal teams are your customers; run discovery, roadmap, onboarding and support.
- **Golden paths:** opinionated, well-supported ways to build, test, deploy and monitor, so teams get speed and safety by default.
- **Self-service:** developers provision what they need (environments, databases, pipelines) without tickets.
- **Metrics:** adoption of the platform, lead time for changes, deployment frequency, change failure rate and time to restore (DORA), developer satisfaction, onboarding time for new engineers.
- **Build vs buy vs adopt open source:** consider total cost, differentiation, skills and risk.
- **AI platforms:** shared model gateway (routing, keys, logging, cost caps), evaluation service, vector store, prompt library, guardrails and monitoring, so product teams do not rebuild them.

Anti-pattern	Fix
Platform team builds what it finds interesting	Discovery with internal users; publish roadmap
Mandating use with no value	Make it the easiest path; measure adoption
No support model	Office hours, SLAs, docs
Too many options	Golden paths and defaults

Key takeaways

- Treat internal users as customers.
- Golden paths and self-service drive adoption.
- Measure with DORA and developer satisfaction.

Practice exercise: Define the roadmap and metrics for an internal AI model gateway.

ADVANCED · CLASS 11

Case studies: payments, communications and open networks

Learn what great platforms do.

Payments APIs (Stripe, Razorpay)

They won on **developer experience**: clean docs, copy-paste quickstarts, test mode, clear error messages, webhooks with signatures and strong reliability. Lesson: reduce time to first transaction and keep the surface area simple.

Communications APIs (Twilio-style)

Usage-based pricing that scales from a hobby project to an enterprise, strong SDKs and tutorials, and expansion across channels (SMS, voice, WhatsApp, email). Lesson: start small with a clear use case, then expand through adjacent APIs.

Open networks (UPI, ONDC)

UPI created an interoperable payments network with open specifications used by many apps; ONDC applies a similar idea to commerce. Lesson: neutral rules, strong governance and low switching costs can create huge ecosystems, but require careful standards, dispute handling and fair economics.

AI APIs

Model APIs compete on quality, price, latency and reliability, and keep adding tools (function calling, batch, caching). Lesson: abstract providers behind a gateway and plan for changes in pricing, limits and model versions.

Key takeaways

- Developer experience is the moat for API products.
- Start with one clear use case, then expand.
- Open networks need governance as much as technology.

Practice exercise: Write a teardown of one API product using the developer funnel and the metrics tree.

ADVANCED · CLASS 12

Case studies: three platform decisions

Practise platform product thinking (illustrative).

Case 1: A payments API has a 40-minute time to first call

- **Diagnosis:** manual account approval, long docs, no sandbox keys at sign-up.
- **Actions:** instant test keys, a 5-minute quickstart, a Postman collection, and sample apps; move approvals to when going live.
- **Target:** TTFC under 10 minutes; activation up 30%.

Case 2: Must remove an old field used by 5% of integrations

- Announce 9 months ahead, add warnings, provide a migration guide and script, contact top 20 users personally, track usage weekly, and delay sunset if critical customers have not migrated.

Case 3: Internal AI gateway for 12 product teams

- Problems: duplicated integrations, no cost visibility, inconsistent safety.
- Platform offers: single API for models, keys per team, caching, rate limits, logging, guardrails, evals and a prompt library.
- Metrics: teams onboarded, time to add a feature, cost per team, incidents, developer satisfaction.

Key takeaways

- Measure and fix the developer funnel.
- Treat deprecation as a launch.
- Internal platforms need product management too.

Practice exercise: Choose a case and write the roadmap for the next 90 days.

ADVANCED · CLASS 13

Interview practice: platform and API scenario questions

Answer platform PM questions with structure.

Q. How would you improve the developer experience of an API?
Model answer: I would measure the developer funnel, find drop-offs, reduce time to first call with instant sandbox keys and a short quickstart, improve error messages and docs, add SDKs and samples, and build support and community channels. I would track TTFC, activation, tickets and developer NPS.
Why it works: Funnel thinking plus metrics.

Q. How do you prioritise between new features and stability?
Model answer: Stability and compatibility are the product for a platform. I would use error budgets and SLOs to decide, protect a share of capacity for reliability and debt, and prioritise features by partner impact and ecosystem growth.
Why it works: Shows platform-specific trade-offs.

Q. How would you price a new API?
Model answer: Use usage-based pricing linked to the value metric with a free tier for experimentation, volume tiers and enterprise contracts. Validate with early customers and watch margin on usage.
Why it works: Ties pricing to value metric.

Q. How do you decide what to open to third parties?
Model answer: Start with capabilities that create partner value and low risk, use scoped access and app review, learn from a small beta group, and expand with strong governance.
Why it works: Controlled openness.

Key takeaways

- Use funnels, SLOs and value metrics.
- Open capabilities gradually.
- Communicate changes early.

Practice exercise: Write your own answers and metrics for each.

ADVANCED · CLASS 14

AI platforms and final project

Apply platform thinking to AI.

- Internal AI platforms: shared model access, evals, guardrails, prompt/skill libraries, cost tracking.
- Exposing your product via MCP so AI assistants can use it.

Final project

Write a platform PRD for a public API or internal AI platform: users, developer journey, API requirements, limits and versioning, partner rules, metrics.

Key takeaways

- AI platforms centralise models, evals and guardrails.
- MCP is a new distribution channel.
- Treat developers like customers.

Practice exercise: Complete the platform PRD.

TOOLKIT

Templates

Platform PRD extras

Field	Fill in
Developer personas	
Jobs & use cases	
Time-to-first-call target	
Endpoints / events	
Auth & scopes	
Limits & versioning	
Partner rules	
Metrics	

TOOLKIT

Cheat sheet

Term / tool	Meaning / use
DX	Developer experience
SDK	Software development kit
Webhook	Event notification sent to your URL
Rate limit	Max requests per time
Deprecation	Planned retirement of an API version
Developer funnel	Discover, sign up, first call, build, go live, scale
Breaking change?	Removing, renaming or changing behaviour
Auth	Keys for simple, OAuth for user data, least privilege
Metrics	TTFC, usage, uptime, developer NPS, ecosystem revenue

Glossary

Idempotency key	Prevents duplicate actions on retries
OAuth	Standard for delegated access
API	Application programming interface
SDK	Software development kit with libraries for an API
OpenAPI	Standard description format for REST APIs
Webhook	Callback sent by a service when an event occurs
OAuth 2.0	Standard for delegated authorisation
Rate limit	Cap on requests in a period
Sandbox	Safe test environment
TTFC	Time to first call
Deprecation	Announced retirement of a feature or version
Idempotency key	Token that makes retries safe
Golden path	Supported default way to build and ship
Take rate	Percentage of transaction value kept by the platform

INTERVIEW PREP

14 interview questions with answers

Q1. What does a platform PM do?
Builds products other teams or developers build on, optimising developer success, reliability and ecosystem growth.

Q2. Key DX metric?
Time to first successful API call, plus time to production.

Q3. How do you version an API?
Explicit versions, backward compatibility, deprecation notices, migration guides and changelogs.

Q4. Platform metrics?
Active developers/apps, success rate, latency, calls, revenue, support friction.

Q5. What is an AI platform team?
Provides shared model access, evals, guardrails, prompt/skill libraries and cost controls to product teams.

Q6. What does a platform PM do differently from an app PM?
Serves developers and partners, focuses on capabilities, stability, developer experience, ecosystem health and versioning, and measures adoption, usage and reliability rather than only end-user features.

Q7. How do you measure developer experience?
Time to first successful call, quickstart completion, docs satisfaction, support tickets per developer, developer NPS and activation rate.

Q8. How do you handle breaking changes?
Avoid them; if needed use versioning, long deprecation notice, migration guides, usage tracking and direct outreach to affected developers.

Q9. REST or GraphQL?
REST is simple, cacheable and widely understood; GraphQL fits complex client data needs. Many platforms offer REST with webhooks and add GraphQL where the use case demands.

Q10. What is idempotency in APIs?
Making repeated requests safe so a retry does not create duplicates, typically using an idempotency key.

Q11. How would you price an API?
Usage-based or tiered, tied to the value metric (messages, calls, transactions), with a free tier for developers and enterprise contracts for scale.

Q12. How do you solve the chicken-and-egg problem for a platform?
Seed with first-party apps, subsidise one side, target a narrow high-value segment first, and make joining easy.

Q13. What are DORA metrics?
Deployment frequency, lead time for changes, change failure rate and time to restore service; used to measure engineering delivery performance.

Q14. How would you run a partner programme?
Define tiers, certification, technical and security requirements, benefits, support and review

processes, and measure partner-sourced revenue and satisfaction.

LEARN MORE

Further reading and tools

- Stripe API docs (example of great DX) ↗
- Razorpay docs ↗
- Postman learning center ↗
- Model Context Protocol ↗
- Stripe API docs (study the DX) ↗
- OpenAPI Initiative ↗
- DORA research ↗
- NPCI UPI overview ↗
- ONDC ↗

© SkillNavora · skillnavora.in · Part of the AI Product Management and GenAI Launchpad programs. Practise with the free AI Tutor, AI PM Toolkit and Practice Lab on skillnavora.in.